Efetividade e Custo de Testes Baseados na Especificação para a Detecção de Erros de Condição de Fronteira em Código Python Gerado por LLMs

João Gabriel Maia da Costa[1], **Lucas Alves Resende**[1]

[1]Departamento de Engenharia de *Software*
Instituto de Ciências Exatas e Informática
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte – MG – Brasil

jgmcosta@sga.pucminas.br, lucas.resende.1475095@sga.pucminas.br

Abstract. *Code generated by Large Language Models (LLMs) is usually accepted when it passes the official tests of each task, which may skip the boundaries of the specification. This work compares specification-based boundary tests, three to five per task, with the official HumanEval and MBPP suites and the extended EvalPlus suites, in effectiveness and executed test cases. In a pilot study with 30 tasks and 270 solutions from three current LLMs, no solution approved by the official tests failed a boundary test (95% upper bound of 11.4%). On 116 boundary mutants, the official suites killed 82.8%, the official plus boundary tests 93.1% and EvalPlus 100%, running 50 times more test cases than the combined suite.*

Resumo. *O código gerado por Modelos de Linguagem de Grande Porte (LLMs) costuma ser aceito quando passa nos testes oficiais de cada tarefa, que podem não exercitar as fronteiras da especificação. Este trabalho compara testes de fronteira baseados na especificação, de três a cinco por tarefa, com as suítes oficiais do HumanEval e do MBPP e as ampliadas do EvalPlus, em efetividade e em casos de teste executados. Em um estudo piloto com 30 tarefas e 270 soluções de três LLMs, nenhuma solução aprovada nos testes oficiais falhou em teste de fronteira (limite superior de 95% de 11,4%). Nos 116 mutantes de fronteira, as suítes oficiais mataram 82,8%, as oficiais com os testes de fronteira 93,1% e o EvalPlus 100%, com 50 vezes mais casos que a suíte combinada.*

Bacharelado em Engenharia de *Software* - PUC Minas
Trabalho de Conclusão de Curso (TCC)

Orientador de conteúdo (TCC I): Leonardo Vilela Cardoso - leonardocardoso@pucminas.br
Orientador (TCC I): Glender Brás de Medeiros - 1386242@sga.pucminas.br
Orientador do TCC II: (A ser definido no próximo semestre)

Belo Horizonte, 11 de novembro de 2026.

1. Introdução

Ferramentas de Inteligência Artificial (IA) passaram a fazer parte da rotina de quem desenvolve *software*. Na pesquisa anual do Stack Overflow de 2025, que ouviu mais

de 49 mil desenvolvedores, 84% declararam usar ou planejar usar essas ferramentas, mas 46% afirmaram não confiar na exatidão do que elas produzem [Stack Overflow 2025]. A queixa mais citada, por 66% dos respondentes, são soluções "quase certas", ou seja, código que parece correto e falha em detalhes. A literatura chega a um diagnóstico parecido. Quando as suítes que avaliam Modelos de Linguagem de Grande Porte (*Large Language Models*, LLMs) foram ampliadas, a taxa de acerto medida caiu em até 28,9%, o que revelou soluções aprovadas que estavam erradas [Liu et al. 2023].

Este trabalho trata de um defeito que se encaixa nesse quadro: o erro de condição de fronteira, isto é, o tratamento incorreto dos valores nos limites de um domínio [White and Cohen 1980]. Um exemplo é implementar "abaixo de t" como "até t". Condições incorretas e casos de borda esquecidos estão entre os defeitos recorrentes em código gerado por LLMs [Wang et al. 2025, Tambon et al. 2025]. O EvalPlus amplia as suítes com entradas geradas automaticamente, sem mirar os limites do enunciado [Liu et al. 2023]. O problema investigado é que **não se sabe se poucos testes de valor-limite, baseados na especificação de cada tarefa, detectam os erros de fronteira que as suítes oficiais deixam passar com a mesma efetividade e a um custo menor que as suítes ampliadas do EvalPlus**. Desse problema decorrem duas perguntas de pesquisa (PP). A PP1 é: quantas soluções de LLMs atuais aprovadas nos testes oficiais falham nos testes de fronteira, e quantas delas o EvalPlus revela? A PP2 é: como os testes de fronteira se comparam às duas suítes em efetividade, medida por teste de mutação, e em custo, medido em casos de teste executados?

Responder a essas perguntas é relevante por três motivos. Primeiro, a análise de valor-limite é conhecida há décadas [Myers et al. 2011], mas as suítes oficiais nem sempre a aplicam: no estudo piloto deste trabalho (Seção 4.9), 24 das 30 tarefas sorteadas, todas com fronteira explícita, não têm nenhuma entrada de fronteira na suíte oficial. Segundo, as suítes ampliadas têm cerca de 80 vezes mais testes que as originais [Liu et al. 2023], o que encarece a execução e dificulta a conferência de cada caso. Terceiro, o oráculo do EvalPlus é a solução de referência, inclusive para entradas que o enunciado não prevê [Liu et al. 2023], enquanto o oráculo dos testes de fronteira vem da própria especificação. Se poucos testes escolhidos a partir da especificação revelarem os mesmos defeitos de fronteira, a avaliação fica mais barata de executar e de auditar, o que contribui para a racionalização de recursos na experimentação de *software*.

O objetivo geral deste trabalho é avaliar a efetividade e o custo de testes de fronteira baseados na especificação, em comparação com as suítes oficiais e do EvalPlus, na detecção de erros de fronteira em código Python gerado por LLMs. Os objetivos específicos são: (i) medir, por LLM e por *benchmark*, a taxa de soluções aprovadas na suíte oficial que falham nos testes de fronteira, e classificar a causa de cada falha revelada; (ii) verificar quantas dessas falhas o EvalPlus revela; (iii) comparar o *mutation score* das suítes oficial, combinada e do EvalPlus, com mutantes de fronteira e genéricos; (iv) comparar o número de casos de teste que cada suíte executa por defeito detectado; e (v) verificar, por reexecução, que os resultados não dependem da ordem dos testes nem da semente de *hash*.

Como resultados, espera-se obter um protocolo reproduzível, a matriz de tarefas elegíveis com seus testes e mutantes e estimativas, com intervalos de confiança, da taxa de falhas não detectadas e da efetividade e do custo de cada suíte. No estudo piloto

(Seção 4.9), nenhuma das 270 soluções aprovadas nos testes oficiais falhou em teste de fronteira, mas o limite superior de 95% (11,4%) não permite afirmar que essa taxa seja baixa. Nos mutantes de fronteira, a suíte combinada matou 93,1% e o EvalPlus 100%, este com 50 vezes mais casos de teste. Na segunda etapa do Trabalho de Conclusão de Curso (TCC II), a triagem de todas as tarefas dos dois *benchmarks* amplia a amostra. Se nenhuma das 64 tarefas já elegíveis revelar falha, o limite superior de 95% da taxa cai de 11,4% para 5,7%.

O restante do texto está organizado da seguinte forma. A Seção 2 apresenta os conceitos que fundamentam o trabalho, e a Seção 3 discute os trabalhos relacionados e a lacuna que esta proposta ocupa. A Seção 4 descreve os materiais e métodos, o estudo piloto já realizado e o cronograma do TCC II. O Apêndice reúne o *prompt*, o resultado por tarefa do estudo piloto e o conteúdo do pacote de replicação.

2. Fundamentação Teórica

2.1. Avaliação de código gerado por LLMs

LLMs são modelos treinados em grandes volumes de texto e código, capazes de produzir uma implementação a partir de uma descrição em linguagem natural [Chen et al. 2021]. Para comparar modelos, a área usa *benchmarks*, conjuntos padronizados de tarefas com enunciado, solução de referência e testes. No HumanEval, cada uma das 164 tarefas traz a assinatura de uma função Python e uma *docstring* com o enunciado e exemplos [Chen et al. 2021]. O MBPP traz enunciados de uma frase e três asserções por tarefa, e sua versão sanitizada, com 427 tarefas, foi revisada manualmente para tornar os enunciados claros e coerentes com os testes [Austin et al. 2021]. O EvalPlus estende os dois com milhares de entradas cujo resultado esperado vem da solução de referência, e origina o HumanEval+ e o MBPP+ [Liu et al. 2023]. Como esses *benchmarks* são públicos desde 2021, suas tarefas podem ter entrado nos dados de treinamento dos modelos, o que pode inflar o desempenho medido; por isso, foram propostos conjuntos com tarefas posteriores ao treinamento [Jain et al. 2024].

2.2. Análise de valor-limite e oráculo de teste

A análise de valor-limite parte da observação de que defeitos se concentram nos limites dos domínios de entrada. Por isso, os casos de teste devem exercitar o próprio valor-limite e os valores imediatamente antes e depois dele [Myers et al. 2011, White and Cohen 1980]. Para cada teste é preciso saber o resultado correto, e o mecanismo que fornece essa resposta é o oráculo [Barr et al. 2015]. Neste trabalho, a especificação de cada tarefa é o seu enunciado, que o *prompt* entrega ao modelo como *Specification* (Apêndice A). Os testes de fronteira são, assim, testes caixa-preta baseados na especificação [Myers et al. 2011]: o oráculo vem do texto do enunciado, e a solução de referência serve para conferi-lo. Por exemplo, a tarefa MBPP/433 pede que se verifique se o número é maior que os elementos do vetor. Uma versão com >= no lugar de > passa nos três testes oficiais, porque nenhum deles usa um número igual ao maior elemento, e o teste de fronteira `check_greater([1, 2, 3], 3) == False` revela o erro.

2.3. Teste de mutação

O teste de mutação avalia a capacidade de uma suíte de detectar defeitos. Operadores de mutação aplicam pequenas alterações sintáticas ao programa e geram versões defeituosas,

chamadas mutantes. Um mutante é "morto" quando ao menos um teste falha sobre ele, e o *mutation score* é a proporção de mutantes mortos entre os válidos [Jia and Harman 2011]. Mutantes equivalentes, que se comportam como o original em todas as entradas válidas, não podem ser mortos e são excluídos do cálculo. A técnica se apoia na hipótese do programador competente, segundo a qual programas reais diferem do correto por pequenos erros, e no efeito de acoplamento, segundo o qual testes que detectam defeitos simples também detectam a maioria dos complexos [DeMillo et al. 1978]. Há ainda evidência de que o *mutation score* se correlaciona com a detecção de defeitos reais [Just et al. 2014]. A troca de operadores relacionais, como < por <=, e o deslocamento de constantes simulam erros de fronteira.

2.4. Testes instáveis e não determinismo

Um teste instável (*flaky test*) é aquele que, para o mesmo código, ora passa, ora falha [Parry et al. 2021]. Entre suas causas estão a dependência de ordem, a aleatoriedade e a iteração sobre coleções sem ordem definida. O Python tem uma fonte própria de não determinismo: a cada processo, o interpretador sorteia uma semente para o *hash* de textos, controlada pela variável `PYTHONHASHSEED` [Python Software Foundation 2026]. Assim, uma função que monta sua saída percorrendo um conjunto de textos pode devolver listas em ordens diferentes em processos diferentes. A técnica mais usada para detectar instabilidade é reexecutar o teste muitas vezes, variando esses fatores, e verificar se o resultado muda [Parry et al. 2021]. Esse fenômeno difere da variabilidade de geração, em que o próprio código produzido pelo LLM muda de uma geração para outra [Ouyang et al. 2025].

3. Trabalhos Relacionados

Os trabalhos relacionados se organizam em três grupos: os que medem a suficiência dos testes usados para avaliar código gerado por LLMs, os que caracterizam defeitos e casos de borda nesse código e os que estudam o não determinismo e a instabilidade de testes. Cada grupo sustenta uma parte desta proposta, e a seção termina com a lacuna que ela pretende ocupar.

O primeiro grupo começa pelo EvalPlus, que amplia as suítes do HumanEval e do MBPP com entradas geradas por LLMs e por mutação de entradas sensível a tipos, verificadas contra a solução de referência [Liu et al. 2023]. O HumanEval+ tem cerca de 80 vezes mais testes que o original e, nos 26 LLMs avaliados, reduz a taxa de acerto (*pass@k*) em até 19,3% a 28,9%. O MuTAP combina LLMs e teste de mutação para gerar testes unitários em Python: os mutantes que sobrevivem são devolvidos ao LLM, que produz novos testes para matá-los [Moradi Dakhel et al. 2024]. Os testes do MuTAP atingem *mutation score* de 93,57% em programas com defeitos sintéticos. Esta proposta usa o EvalPlus como linha de base e reutiliza do MuTAP a ideia de medir a capacidade de detecção de uma suíte por meio de mutantes.

No segundo grupo, um estudo analisa 333 defeitos em código gerado por três LLMs e organiza uma taxonomia de dez padrões, entre eles o caso de borda esquecido [Tambon et al. 2025]. Outro classifica manualmente 557 soluções incorretas de seis LLMs para o HumanEval, e as condições incorretas estão entre os erros mais frequentes nos seis modelos [Wang et al. 2025]. Como os dois estudos usam modelos anteriores

aos avaliados aqui, esta proposta testa se o erro de fronteira persiste nos LLMs atuais. A inspeção de 114 tarefas em que seis LLMs de 2025 falham, em quatro *benchmarks*, encontra quatro padrões de fraqueza, entre eles o tratamento incorreto de casos de borda, em resultado ainda em versão *preprint* [Molzam Sharifloo et al. 2025]. Como quase nenhuma tarefa do HumanEval e do MBPP derrota todos os modelos nesse estudo, esta proposta procura os defeitos restantes nas soluções aprovadas.

Ainda no segundo grupo, dois trabalhos testam fronteiras com apoio de LLMs. No primeiro, o Claude Sonnet 4 gera testes baseados em exemplos e em propriedades para 16 tarefas do HumanEval cuja referência falha no HumanEval+ [Tanaka et al. 2025]. Cada abordagem detecta 68,75% dos defeitos, e a combinação detecta 81,25%. Os testes por exemplo são melhores em condições de fronteira específicas, mas 12 dos 16 conjuntos têm testes com oráculo errado. No segundo, entradas de valor-limite geradas pelo GPT superam a análise de valor-limite tradicional em seis de oito programas C/C++, e aumentar o número de entradas não melhora a detecção de forma consistente [Guo et al. 2025]. Os dois resultados, ainda em versão *preprint*, apoiam o uso de poucos testes de fronteira com oráculo revisado pelos autores.

No terceiro grupo, um estudo gera várias respostas do ChatGPT para 829 problemas e, no HumanEval, 47,56% das tarefas não têm nenhum par de respostas com saídas de teste iguais [Ouyang et al. 2025]. Outro executa repetidamente as suítes de 22.352 projetos Python e encontra 7.571 testes instáveis, com predomínio da dependência de ordem (59%) [Gruber et al. 2021]. Os autores estimam que 170 reexecuções dão 95% de confiança de que um teste aprovado não é instável. A análise de 201 correções de testes instáveis em 51 projetos de código aberto também aponta a dependência de ordem entre as causas mais frequentes [Luo et al. 2014]. Com base nesses estudos, esta proposta gera várias soluções por tarefa e reexecuta cada solução congelada variando a ordem dos testes e a semente de *hash*.

Em conjunto, esses trabalhos mostram que as suítes originais aprovam soluções erradas, que casos de borda estão entre os defeitos do código gerado por LLMs e que testes focados na fronteira têm valor próprio. Nenhum deles, porém, compara testes de valor-limite baseados na especificação com as suítes ampliadas do EvalPlus, em detecção e em custo, sobre soluções de LLMs atuais. O mais próximo compara entradas de valor-limite com a análise tradicional [Guo et al. 2025], mas em programas C/C++ e sem uma suíte ampliada como referência. Essa é a lacuna que este trabalho ocupa.

4. Materiais e Métodos

4.1. Tipo de pesquisa

Quanto à natureza, trata-se de pesquisa aplicada, pois busca reduzir o custo de avaliar código gerado por LLMs; quanto aos objetivos, de pesquisa descritiva; e quanto à abordagem, de pesquisa quantitativa [Wazlawick 2014]. A comparação entre suítes é um experimento controlado, porque a suíte é o único fator que varia sobre as mesmas soluções e mutantes [Wohlin et al. 2012]. O fator tem quatro níveis (oficial, de fronteira, combinada e EvalPlus), e as variáveis de resposta são a taxa de falhas não detectadas, o *mutation score* e o custo. A hipótese nula principal é que a suíte combinada e o EvalPlus matam a mesma proporção de mutantes de fronteira. A comparação entre modelos é

observacional, pois eles diferem em aspectos não manipulados. A Figura 1 apresenta as etapas.

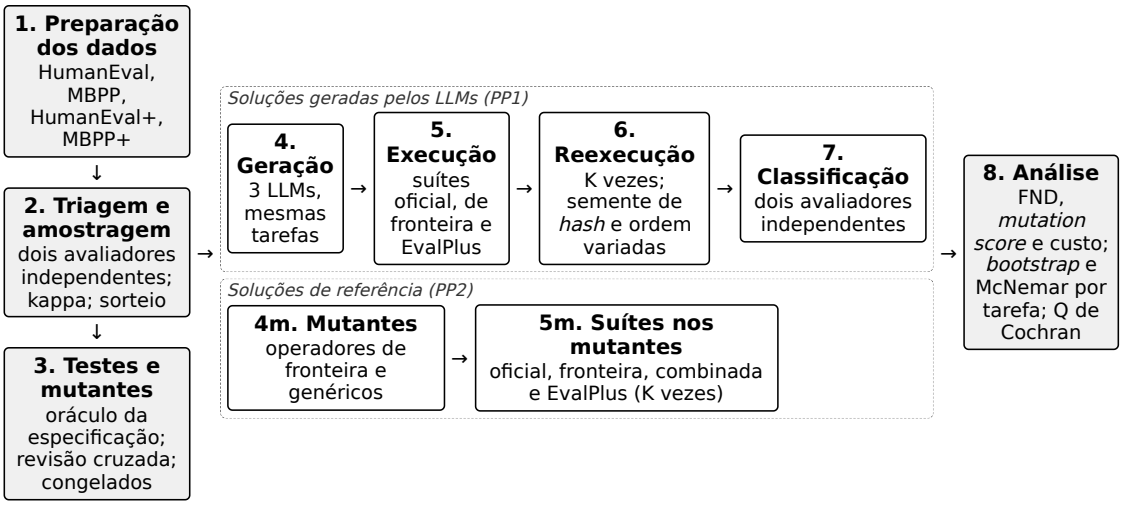


Figura 1. Visão geral da metodologia

4.2. Ambiente e dados

O experimento usa Python 3.13.18, a linguagem das tarefas, e o executor de testes *pytest* 9.2, com os complementos *pytest-randomly*, que embaralha a ordem dos testes, e *pytest-timeout*, que limita o tempo de cada teste. As execuções ocorrem em contêiner Linux isolado, com 4 núcleos virtuais Intel Xeon de 2,3 GHz e 16 GB de memória, sem acesso à rede. Os dados são o HumanEval, o MBPP sanitizado e as suítes do EvalPlus nas versões HumanEval+ v0.1.10, com 164 tarefas, e MBPP+ v0.2.0, com 378 tarefas, cada arquivo fixado por *hash* SHA-256.

4.3. Triagem e amostragem das tarefas

Uma tarefa é elegível se: (1) o enunciado especifica um limite, intervalo, valor extremo ou transição entre condições; (2) existe entrada válida que exercita essa fronteira; (3) o resultado esperado na fronteira é determinável pelo enunciado; e (4) a solução de referência corrigida pelo EvalPlus concorda com o enunciado nos testes de fronteira. Quando a referência diverge do enunciado, a tarefa é excluída e a divergência é registrada, para que nenhum oráculo dependa de uma escolha dos autores.

Uma pré-triagem automática, por busca de 25 termos como *below*, *at most* e *between* (Apêndice C), identificou 177 tarefas candidatas. Destas, 14 do MBPP não existem no MBPP+, o que deixa 163: 86 do HumanEval e 77 do MBPP. Cada aluno avalia os critérios 1 a 3 em todas as candidatas, de forma independente, sem assistentes de IA na decisão; o único apoio foi um tradutor automático para a leitura dos enunciados. A concordância é medida pelo kappa de Cohen [Cohen 1960], e as divergências são resolvidas por consenso. Entre as elegíveis, sorteiam-se 30 tarefas com a semente 20261109, 15 de cada *benchmark*, número limitado pelo orçamento de geração do piloto. O critério 4 é verificado nas sorteadas, ao conferir cada teste de fronteira contra a referência. Como a busca por termos pode perder fronteiras implícitas, as conclusões se restringem a fronteiras explícitas.

4.4. Testes de fronteira e mutantes

Para cada fronteira, os testes cobrem o valor-limite e um vizinho de cada lado, sempre que o domínio permite [Myers et al. 2011]. Cada teste registra a entrada, a saída esperada e o trecho do enunciado que fundamenta o oráculo. Os testes foram rascunhados com o assistente Claude Opus 5.5 (Anthropic), o mesmo modelo avaliado na geração. Cada aluno conferiu, sem apoio de IA, se a entrada exercita a fronteira e se a saída decorre do trecho citado, e os testes foram congelados antes da geração das soluções. Os mutantes são aplicados às referências por um mutador próprio, escrito com o módulo `ast` do Python, que aplica só os operadores de interesse e registra o grupo de cada mutante. Os operadores de fronteira trocam comparações estritas e inclusivas ($< \leftrightarrow <=$, $> \leftrightarrow >=$) e deslocam em ± 1 constantes e limites de `range`. Os genéricos trocam operadores aritméticos ($+ \leftrightarrow -$) e lógicos (`and` $\leftrightarrow$ `or`) e removem negações.

Os dois grupos são relatados separadamente, porque os testes de fronteira miram os mesmos limites que os operadores de fronteira. Um mutante é válido quando compila e difere da referência, e o teste que esgota o limite de 10 s conta como falha. Um mutante é equivalente quando nenhuma entrada válida pelo enunciado o distingue da referência. São válidas as entradas que o enunciado não restringe, como zero e negativos, e não o são as que o contradizem ou cujo resultado ele não determina. Os sobreviventes foram analisados com apoio de IA, com a justificativa de cada um registrada no pacote; no TCC II, os dois alunos fazem essa análise de forma independente.

4.5. Geração das soluções

Cada modelo recebe o enunciado e um *prompt* padronizado que pede apenas a implementação da função (Apêndice A). No HumanEval, o enunciado é a assinatura com a *docstring*. No MBPP, é o enunciado do MBPP+, que acrescenta à frase original uma asserção de exemplo e assim fixa o nome da função e o tipo do retorno [Liu et al. 2023]. Essa asserção permanece na suíte oficial, como no MBPP+. Cada geração é uma chamada nova, sem histórico. Um *script* extrai o código sem correção, verifica sintaxe e assinatura e congela a solução com seu *hash*. Uma resposta truncada pelo limite de *tokens* antes de conter código é refeita uma vez com 65.536 *tokens*; se ainda não houver código válido, conta como falha de geração.

O estudo piloto usa três LLMs de fornecedores diferentes, todos pelas Interfaces de Programação de Aplicações (APIs, do inglês *Application Programming Interfaces*) oficiais: o Claude Opus 5.5 (`claude-opus-5-5`), o GPT-6.1 Sol (`gpt-6.1-sol`) e o DeepSeek V4 Pro (`deepseek-v4-pro`). A temperatura não é enviada, e o nível de raciocínio é o padrão de cada fornecedor, como no uso cotidiano dessas ferramentas. Como esse nível difere entre os fornecedores, a comparação entre modelos é observacional, enquanto a comparação entre suítes, foco deste trabalho, é controlada. O limite é de 16.384 *tokens* de saída. De cada resposta ficam registrados o identificador devolvido, os parâmetros, os *tokens*, a data e o tempo de geração. Cada modelo gera três soluções para cada uma das 30 tarefas, o que totaliza 270 gerações, feitas em 9 de novembro de 2026.

4.6. Execução das suítes e reexecução

Cada solução congelada é executada contra a suíte oficial, os testes de fronteira e a suíte do EvalPlus. Na oficial, cada asserção é um caso de teste; no EvalPlus, as entradas são

verificadas dentro de um único caso do *pytest*, mas cada entrada conta como um caso na métrica de custo. Cada sujeito (solução, referência ou mutante) é executado K vezes, cada vez em um processo Python novo, com valor aleatório de `PYTHONHASHSEED`, ordem dos testes embaralhada com semente registrada e limite de 10 s por teste. Esses controles cobrem as causas de instabilidade aplicáveis a funções puras: ordem, iteração sobre conjuntos, aleatoriedade e tempo.

Ao variar a semente e a ordem, o protocolo altera de propósito fatores que a definição clássica de teste instável mantém fixos [Parry et al. 2021]. Um par (solução, teste) instável indica, portanto, dependência de algo que o enunciado não especifica, e cada caso é inspecionado. Cada par é classificado como estável-passa, estável-falha ou instável. O estudo piloto e o TCC II usam $K = 10$, e cada par instável é inspecionado. A reexecução é um controle de validade, e não uma pergunta de pesquisa: ela garante que a FND e o *mutation score* não refletem ruído de execução. Por isso, o protocolo não adota as 170 reexecuções que dariam 95% de confiança de que um teste aprovado não é instável [Gruber et al. 2021], e um par estável nas 10 execuções é tratado como estável.

4.7. Métricas e análise estatística

A suíte oficial é a que acompanha a tarefa, e a suíte combinada acrescenta a ela os testes de fronteira. A métrica principal, que responde à PP1, é a taxa de falhas não detectadas (FND) por uma suíte S, oficial ou do EvalPlus (Equação 1). A PP2 usa o *mutation score* de cada suíte (Equação 2), calculado separadamente para mutantes de fronteira e genéricos, e o custo (Equação 3). Um caso de teste é uma entrada com saída esperada, e o custo é calculado nas tarefas que têm mutantes do grupo.

$$\text{FND}_S = \frac{\text{soluções aprovadas em } S \text{ que falham por erro de fronteira}}{\text{soluções aprovadas de forma estável em } S} \times 100 \qquad (1)$$

$$\textit{Mutation score}_S = \frac{\text{mutantes válidos e não equivalentes mortos por } S}{\text{mutantes válidos e não equivalentes}} \times 100 \qquad (2)$$

$$\text{Custo}_S = \frac{\text{casos de teste da suíte } S}{\text{mutantes mortos por } S} \qquad (3)$$

Toda falha revelada por uma suíte e não pela oficial é classificada pelos dois alunos, de forma independente, como erro de fronteira, defeito real fora das fronteiras, entrada não especificada pelo enunciado ou entrada que o contradiz, com a mesma regra de entrada válida usada para os mutantes. Como as gerações de uma mesma tarefa não são independentes, os intervalos de confiança de 95% são obtidos por *bootstrap* com reamostragem de tarefas [Efron and Tibshirani 1993, Field and Welsh 2007]. Quando nenhum evento é observado, reporta-se o limite superior de Wilson calculado com o número de tarefas, o que é conservador [Wilson 1927].

As suítes são comparadas no nível da tarefa, pelo teste exato de McNemar [McNemar 1947], sobre as tarefas em que só uma das suítes detecta algum defeito ou mutante. Os modelos são comparados pelo teste Q de Cochran [Cochran 1950], sobre o indicador, por tarefa, de que ao menos uma solução do modelo é FND; sem eventos, a comparação é só descritiva. Os *benchmarks* são descritos separadamente. Como um teste não significativo não demonstra igualdade, a diferença de *mutation score* entre as suítes

também é relatada com intervalo *bootstrap* por tarefa. O nível de significância é de 5%, com correção de Holm em cada família de comparações [Holm 1979]. A sensibilidade à instabilidade é avaliada recalculando a FND com as soluções instáveis excluídas, contadas como falha e como aprovação.

4.8. Instrumentos

Os instrumentos são *scripts* Python escritos pelos autores, um para cada etapa da Figura 1, com a biblioteca SciPy 1.18 nos testes estatísticos. Todos os instrumentos, dados brutos e resultados formam o pacote de replicação (Apêndice C), disponível em `https://github.com/Joaogabrielmaia/TCC-Piloto`.

4.9. Estudo piloto

No estudo piloto, os alunos concordaram na triagem em 125 das 163 candidatas ($\kappa = 0{,}49$, concordância moderada) e resolveram as 38 divergências por consenso, o que resultou em 64 tarefas elegíveis, 44 do HumanEval e 20 do MBPP. Os 114 testes de fronteira das 30 tarefas sorteadas passaram na referência, de modo que o critério 4 não excluiu nenhuma, e a revisão cruzada teve uma divergência. As três respostas do DeepSeek para o MBPP/239 esgotaram o limite de *tokens* com raciocínio e foram refeitas com 65.536; as demais vieram com código válido na primeira tentativa, e os identificadores devolvidos coincidiram com os solicitados.

As 270 soluções correspondem a 181 códigos distintos: 54 do Claude, 58 do GPT e 75 do DeepSeek, com alguns códigos idênticos entre modelos. Na PP1, as 270 soluções passaram de forma estável na suíte oficial, e nenhuma falhou em teste de fronteira: $\text{FND}_{\text{oficial}} = 0$ de 270, com limite superior de Wilson de 11,4% sobre as 30 tarefas, e $\text{FND}_{\text{EvalPlus}} = 0$ de 255. O EvalPlus reprovou 15 soluções aprovadas pelas outras suítes (3 do Claude, 9 do GPT e 3 do DeepSeek), concentradas em 4 tarefas. Na classificação dos alunos (concordância em 11 de 15, $\kappa = 0{,}59$, com consenso), 6 vêm de entradas que contradizem o enunciado (lista vazia com tamanho 1, MBPP/223) e 8 de entradas cujo resultado ele não determina (número não inteiro no MBPP/244 e intervalo invertido no MBPP/589). Só 1 é defeito real, fora das fronteiras (negativos no HumanEval/122), em uma tarefa (McNemar por tarefa, $p = 1$).

Dos 180 mutantes, 128 de fronteira e 52 genéricos, os 14 que sobreviveram a todas as suítes são equivalentes (12 de fronteira e 2 genéricos), e cinco tarefas não têm ponto de mutação de fronteira. Nos 116 mutantes de fronteira restantes, de 25 tarefas, a suíte oficial matou 82,8%, os testes de fronteira 85,3%, a combinada 93,1% e o EvalPlus 100% (Tabela 2, Apêndice B). Os testes de fronteira mataram 12 mutantes que a oficial deixou passar. Os 8 mortos só pelo EvalPlus estão em 3 tarefas (McNemar por tarefa, $p = 0{,}25$): seis exercitam zero ou negativos no MBPP/244 e no MBPP/589, fronteiras implícitas que o enunciado não cita, e os outros dois, posições da grade do HumanEval/129. Nos 50 genéricos, a oficial matou 96,0%, e a combinada e o EvalPlus mataram todos.

Nas 25 tarefas com mutantes de fronteira, a suíte combinada tem 231 casos de teste, pois 6 dos 97 testes de fronteira repetem entradas da oficial, e o EvalPlus tem 11.471. O custo por mutante de fronteira morto foi de 1,5 caso de teste na oficial, 1,0 nos testes de fronteira, 2,1 na combinada e 98,9 no EvalPlus. O EvalPlus executou, portanto, 50 vezes mais casos que a combinada para matar 6,9 pontos percentuais a mais

de mutantes. Se os 12 sobreviventes de fronteira não fossem equivalentes, os escores cairiam para 75,0%, 77,3%, 84,4% e 90,6%, sem mudar a ordem das suítes. A conta não inclui o tempo de execução nem o esforço manual de escrever e revisar os testes, que o TCC II passa a registrar. Na reexecução, os 2.547 pares (solução, teste oficial ou de fronteira) mantiveram o resultado nas 10 execuções, e nenhum dos 51.270 resultados de teste, incluindo EvalPlus, referências e mutantes, foi instável.

O estudo piloto não encontrou erro de fronteira explícita nos três LLMs avaliados, mas o limite de 11,4% não permite afirmar que esses erros sejam raros, e o resultado pode refletir a exposição dos modelos às tarefas no treinamento [Jain et al. 2024]. A suíte combinada ficou 6,9 pontos percentuais abaixo do EvalPlus com 1/50 dos casos, e a diferença está em fronteiras implícitas, que os testes não cobrem por construção. Além disso, 14 das 15 reprovações do EvalPlus vieram de entradas que o enunciado contradiz ou não determina, o que mostra o custo de usar a solução de referência como oráculo. O TCC II, portanto, amplia o número de tarefas e passa a medir o tempo e o esforço manual.

4.10. Etapas e cronograma do TCC II

O TCC II usa todas as tarefas elegíveis após a triagem manual das 542 tarefas do HumanEval+ e do MBPP+, sem a pré-triagem por termos. Com as 64 já elegíveis e nenhuma falha, o limite superior de 95% da FND seria de 5,7%, e cada tarefa adicional o reduz. Cada modelo gera 20 soluções por tarefa, no mínimo 3.840 gerações, o que permite estimar a taxa de aprovação de cada modelo por tarefa. As etapas são: (1) triagem, testes e mutantes; (2) geração; (3) execução, reexecução e mutação; (4) classificação das falhas e análise; e (5) escrita. Na Tabela 1, cada aluno tria e testa um *benchmark*, faz a triagem independente do outro para o kappa e executa as suítes nas soluções que gerou. Assim, as tarefas de um aluno dependem das do outro em dois pontos: a execução (J3 e L3) usa os testes e mutantes dos dois *benchmarks*, e por isso J1 e L1 a precedem, e a classificação das falhas é feita pelos dois em separado.

Tabela 1. Cronograma quinzenal do TCC II por aluno (fev. a jun. de 2027)

Tarefa	Fev		Mar		Abr		Mai		Jun	
	1ª	2ª	1ª	2ª	1ª	2ª	1ª	2ª	1ª	2ª
João Gabriel Maia da Costa										
J1. Triagem e testes do HumanEval+; triagem do MBPP+	■	■	■							
J2. Geração com o Claude e o GPT				■	■					
J3. Execução das suítes nas soluções do Claude e do GPT					■	■				
J4. Classificação das falhas (independente)							■			
J5. Análise da FND e comparação entre modelos							■	■		
J6. Escrita dos resultados e pacote de replicação								■	■	■
Lucas Alves Resende										
L1. Triagem e testes do MBPP+; triagem do HumanEval+	■	■	■							
L2. Geração com o DeepSeek; mutantes da escala final				■	■					
L3. Execução nas soluções do DeepSeek; reexecução					■	■				
L4. Classificação das falhas (independente)							■			
L5. Comparação entre suítes, custo e tempo							■	■		
L6. Escrita das ameaças à validade e da conclusão								■	■	■

Referências

Austin, J., Odena, A., Nye, M., Bosma, M., et al. (2021). Program synthesis with large language models. arXiv preprint arXiv:2108.07732.

Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., and Yoo, S. (2015). The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525.

Chen, M., Tworek, J., Jun, H., Yuan, Q., et al. (2021). Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374.

Cochran, W. G. (1950). The comparison of percentages in matched samples. *Biometrika*, 37(3/4):256–266.

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46.

DeMillo, R. A., Lipton, R. J., and Sayward, F. G. (1978). Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41.

Efron, B. and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall, New York.

Field, C. A. and Welsh, A. H. (2007). Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B*, 69(3):369–390.

Gruber, M., Lukasczyk, S., Krois, F., and Fraser, G. (2021). An empirical study of flaky tests in Python. In *Proceedings of the 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 148–159. IEEE.

Guo, X., Li, C., and Tsuchiya, T. (2025). Boundary value test input generation using prompt engineering with LLMs: Fault detection and coverage analysis. arXiv preprint arXiv:2501.14465.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.

Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. (2024). LiveCodeBench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974.

Jia, Y. and Harman, M. (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678.

Just, R., Jalali, D., Inozemtseva, L., Ernst, M. D., Holmes, R., and Fraser, G. (2014). Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 654–665. ACM.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. (2023). Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 21558–21572.

Luo, Q., Hariri, F., Eloussi, L., and Marinov, D. (2014). An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 643–653. ACM.

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Molzam Sharifloo, A., Heydari, M., Kazerooni, P., Maninger, D., and Mezini, M. (2025). Where do LLMs still struggle? an in-depth analysis of code generation benchmarks. arXiv preprint arXiv:2511.04355.

Moradi Dakhel, A., Nikanjam, A., Majdinasab, V., Khomh, F., and Desmarais, M. C. (2024). Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology*, 171:107468.

Myers, G. J., Sandler, C., and Badgett, T. (2011). *The Art of Software Testing*. John Wiley & Sons, Hoboken, NJ, 3 edition.

Ouyang, S., Zhang, J. M., Harman, M., and Wang, M. (2025). An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2):42:1–42:28.

Parry, O., Kapfhammer, G. M., Hilton, M., and McMinn, P. (2021). A survey of flaky tests. *ACM Transactions on Software Engineering and Methodology*, 31(1):17:1–17:74.

Python Software Foundation (2026). Command line and environment: PYTHONHASHSEED. `https://docs.python.org/3/using/cmdline.html`. Acesso em: 8 nov. 2026.

Stack Overflow (2025). 2025 developer survey: AI. `https://survey.stackoverflow.co/2025/ai`. Acesso em: 8 nov. 2026.

Tambon, F., Moradi Dakhel, A., Nikanjam, A., Khomh, F., Desmarais, M. C., and Antoniol, G. (2025). Bugs in large language models generated code: An empirical study. *Empirical Software Engineering*, 30(3):65.

Tanaka, H., Tanaka, H., Shimari, K., and Matsumoto, K. (2025). Understanding the characteristics of LLM-generated property-based tests in exploring edge cases. arXiv preprint arXiv:2510.25297.

Wang, Z., Zhou, Z., Song, D., Huang, Y., Chen, S., Ma, L., and Zhang, T. (2025). Towards understanding the characteristics of code generation errors made by large language models. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, pages 2587–2599, Ottawa, Canada. IEEE.

Wazlawick, R. S. (2014). *Metodologia de Pesquisa para Ciência da Computação*. Elsevier, Rio de Janeiro, 2 edition.

White, L. J. and Cohen, E. I. (1980). A domain strategy for computer program testing. *IEEE Transactions on Software Engineering*, SE-6(3):247–257.

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg, 1 edition.

Apêndice

A. *Prompt* Utilizado para Geração das Soluções

O *prompt* abaixo é enviado ao modelo em cada geração. O marcador `{task}` é substituído pelo enunciado da tarefa: a assinatura com a *docstring*, no HumanEval, ou

o enunciado do MBPP+ com sua asserção de exemplo, no MBPP. O *prompt* de sistema é: *"You write Python code. Answer only with the requested code block."*

Prompt 1: Prompt para geração das soluções

````
You are an experienced Python programmer.

Implement the Python function described below so that it
fully satisfies the specification.

Rules:
1. Return only the complete function definition, including
   its signature and any imports it needs, inside a single
   ```python code block.
2. Keep the function name and parameters exactly as given.
3. Use only the Python standard library.
4. Do not include tests, usage examples, print statements,
   or explanations.

Specification:
{task}
````

B. Resultado por Tarefa do Estudo Piloto

A Tabela 2 resume, para cada uma das 30 tarefas sorteadas, o número de casos de teste de cada suíte, quantos testes de fronteira já constam da suíte oficial (entre parênteses), o resultado das nove soluções de cada tarefa (três por modelo) e os mutantes de fronteira não equivalentes mortos por cada suíte. Todas as 270 soluções passaram na suíte oficial. Um mutante é considerado morto quando ao menos um teste da suíte falha nele em todas as K execuções, e "–" indica tarefa sem ponto de mutação de fronteira na referência. Os 114 testes, com entrada, saída esperada e justificativa, e os 180 mutantes estão no pacote de replicação.
````

Tabela 2. Resultado por tarefa do estudo piloto (9 soluções por tarefa)

	Casos de teste			Soluções		Mutantes de fronteira			
Tarefa	**Oficial**	**Fronteira**	**EvalPlus**	**Falham front.**	**Reprov. EvalPlus**	**Válidos**	**Oficial**	**Comb.**	**EvalPlus**
HumanEval/0	7	3 (0)	1.006	0	0	3	2	3	3
HumanEval/12	3	4 (0)	936	0	0	–	–	–	–
HumanEval/21	5	3 (0)	1.004	0	0	–	–	–	–
HumanEval/30	4	4 (0)	1.003	0	0	3	3	3	3
HumanEval/68	10	4 (0)	1.008	0	0	5	4	5	5
HumanEval/72	6	4 (0)	697	0	0	1	1	1	1
HumanEval/80	8	3 (1)	949	0	0	5	5	5	5
HumanEval/96	10	5 (2)	190	0	0	7	7	7	7
HumanEval/108	9	4 (0)	1.008	0	0	3	3	3	3
HumanEval/122	5	4 (0)	1.005	0	1	3	3	3	3
HumanEval/126	13	4 (1)	1.012	0	0	3	3	3	3
HumanEval/129	11	4 (0)	199	0	0	18	16	16	18
HumanEval/136	12	4 (0)	1.009	0	0	6	6	6	6
HumanEval/156	15	4 (2)	220	0	0	–	–	–	–
HumanEval/163	5	4 (0)	986	0	0	6	4	6	6
MBPP/66	3	4 (0)	110	0	0	3	1	3	3
MBPP/167	3	5 (1)	115	0	0	2	2	2	2
MBPP/223	4	4 (0)	104	0	6	1	0	1	1
MBPP/238	3	3 (0)	105	0	0	–	–	–	–
MBPP/239	3	4 (0)	107	0	0	11	11	11	11
MBPP/244	3	3 (1)	112	0	4	3	0	0	3
MBPP/256	3	5 (0)	104	0	0	9	8	9	9
MBPP/277	3	3 (0)	109	0	0	1	1	1	1
MBPP/283	3	4 (0)	107	0	0	1	1	1	1
MBPP/388	3	4 (0)	106	0	0	1	1	1	1
MBPP/433	3	3 (0)	106	0	0	1	0	1	1
MBPP/566	3	3 (0)	116	0	0	–	–	–	–
MBPP/589	3	3 (0)	106	0	4	7	3	4	7
MBPP/603	3	4 (0)	106	0	0	5	5	5	5
MBPP/745	3	4 (0)	107	0	0	8	6	8	8
Total	**169**	**114 (8)**	**13.852**	**0**	**15**	**116**	**96**	**108**	**116**

C. Pacote de Replicação

O pacote de replicação, disponível em `https://github.com/Joaogabrielmaia/TCC-Piloto`, contém os arquivos de dados com seus *hashes* SHA-256, a decisão de triagem de cada tarefa, os testes de fronteira com o trecho do enunciado que justifica cada oráculo, os *scripts* de preparação, geração, extração, mutação, execução e análise, as respostas brutas dos modelos, as soluções congeladas, os mutantes com a justificativa de cada equivalente, o resultado de cada execução de testes, com as sementes usadas, e as métricas calculadas.

A pré-triagem buscou nos enunciados os 25 termos a seguir: *above*, *at least*, *at most*, *below*, *between*, *empty*, *greater than*, *inclusive*, *larger than*, *largest*, *less than*, *limit*, *lower*, *maximum*, *minimum*, *more than*, *negative*, *non-negative*, *positive*, *range*, *smaller than*, *smallest*, *threshold*, *upper* e *zero*. O custo da Seção 4.9 soma, nas 25 tarefas com mutantes de fronteira da Tabela 2, os casos de teste de cada suíte (140 na oficial, 97 nos testes de fronteira e 11.471 no EvalPlus) e os divide pelos mutantes mortos (96, 99 e 116; 108 na combinada, com 231 casos, sem as 6 entradas repetidas da oficial).